

Reviewer Pack — Tropicalized Homology of Graphs vs Circuit Lower Bounds for ...

Entry 1e7b5e62f37d · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-23 01:59:13 UTC

Tropicalized Homology of Graphs vs Circuit Lower Bounds for Tropical Arithmetic

Entry ID: 1e7b5e62f37d **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-23 01:59:13 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Topology (Tropicalization) **Field B** (complexity object): Complexity Theory: Circuit Lower Bounds for Tropical Arithmetic

Statement:

[‘For any graph G with n vertices, the number of distinct tropical homology classes of G is upper-bounded by a function $f(n)$ that depends polynomially on n .’, ‘The complexity-theoretic corollary is that there exists an arithmetic circuit computing a tropical function over G with size $O(f(n))$.’, ‘For all instances with $n \leq 40$, the number of distinct tropical homology classes of G is exactly equal to the size of the smallest arithmetic circuit computing a tropical function over G .’]

Rationale (proposer’s reasoning):

[‘Tropicalization provides a way to translate geometric and topological properties into algebraic ones, which can be analyzed through arithmetic circuits.’, ‘The conjecture suggests that the complexity of tropical arithmetic operations is related to the topological structure of graphs, potentially revealing new insights into circuit lower bounds.’, ‘By establishing a connection between tropicalized homology and circuit size, we could uncover novel techniques for proving circuit lower bounds.’]

Taxonomy category: TROPICAL_FOURIER_ANALYSIS (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): ae85334cdd813820

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Determine if the number of distinct tropical homology classes (TropClasses) matches the size of the smallest arithmetic circuit (CircuitSize) within a polynomial bound for graphs G with $n \leq 40$, where $\text{TropClasses} \leq \text{CircuitSize} * f(n)$

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - tropicalized homology graphs AND circuit lower bounds tropical arithmetic - algebraic topology tropicalization AND complexity theory circuit lower bounds tropical arithmetic - distinct tropical homology classes graph n vertices polynomially bounded AND $O(f(n))$ arithmetic circuit size

Top relevant hits considered: - [<http://arxiv.org/abs/2603.19020v1>] GWTC-4.0: Tests of General Relativity. II. Parameterized Tests - [<http://arxiv.org/abs/2010.14529v3>] Tests of General Relativity with Binary Black Holes from the second LIGO-Virgo Gravitational-Wave Transient Catalog - [<http://arxiv.org/abs/2403.03004v1>] Ultralight vector dark matter search using data from the KAGRA O3GK run

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def generate_random_graph(n):
    G = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(i + 1, n):
            if random.choice([True, False]):
                G[i][j] = G[j][i] = random.randint(1, 10)
    return G

def is_tropicalizable(G):
    n = len(G)
    for i in range(n):
        for j in range(i + 1, n):
            if G[i][j] == 0:
                return False
    return True

def tropicalize_graph(G):
    n = len(G)
    G_trop = [[math.inf] * n for _ in range(n)]
```

```

for i in range(n):
    G_trop[i][i] = 0
for i in range(n):
    for j in range(i + 1, n):
        if G[i][j] != 0:
            G_trop[i][j] = G[j][i] = min(G_trop[i][k] + G[k][j] for k in range(n) if k != i and k != j)
return G_trop

def compute_tropical_homology_classes(G_trop):
    n = len(G_trop)
    visited = [False] * n
    homology_classes = 0

    def dfs(node, path):
        nonlocal homology_classes
        if node in path:
            homology_classes += 1
            return True
        visited[node] = True
        for neighbor in range(n):
            if G_trop[node][neighbor] != math.inf and not visited[neighbor]:
                if dfs(neighbor, path + [node]):
                    return True
        visited[node] = False
        return False

    for i in range(n):
        if not visited[i]:
            dfs(i, [])

    return homology_classes

def construct_arithmetic_circuit(G_trop):
    n = len(G_trop)
    circuit_size = 0

    def find_min_path(start, end):
        nonlocal circuit_size
        min_dist = math.inf
        for k in range(n):
            if G_trop[start][k] != math.inf and G_trop[k][end] != math.inf:
                dist = G_trop[start][k] + G_trop[k][end]
                if dist < min_dist:
                    min_dist = dist
                    circuit_size += 2
        return min_dist

    for i in range(n):
        for j in range(i + 1, n):
            find_min_path(i, j)

    return circuit_size

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.choice([5, 10, 15, 20, 30, 40])

```

```

G = generate_random_graph(n)

if not is_tropicalizable(G):
    return {
        "metric_name": "Tropicalized Homology Size / Circuit Size",
        "metric_value": None,
        "instances_tested": 1,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

G_trop = tropicalize_graph(G)
TropClasses = compute_tropical_homology_classes(G_trop)
CircuitSize = construct_arithmetic_circuit(G_trop)

return {
    "metric_name": "Tropicalized Homology Size / Circuit Size",
    "metric_value": TropClasses,
    "instances_tested": 1,
    "conjecture_holds": TropClasses == CircuitSize,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results if r["metric_value"] is not None) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results if r["metric_value"] is not None) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

fined'}
TRIAL: {'metric_name': 'Tropicalized Homology Size / Circuit Size', 'metric_value': None, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': 'mapping_undefined'}
TRIAL: {'metric_name': 'Tropicalized Homology Size / Circuit Size', 'metric_value': None, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': 'mapping_undefined'}
TRIAL: {'metric_name': 'Tropicalized Homology Size / Circuit Size', 'metric_value': None, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': 'mapping_undefined'}
TRIAL: {'metric_name': 'Tropicalized Homology Size / Circuit Size', 'metric_value': None, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': 'mapping_undefined'}

```

```

TRIAL: {'metric_name': 'Tropicalized Homology Size / Circuit Size', 'metric_value': None, 'instances_
TRIAL: {'metric_name': 'Tropicalized Homology Size / Circuit Size', 'metric_value': None, 'instances_
TRIAL: {'metric_name': 'Tropicalized Homology Size / Circuit Size', 'metric_value': None, 'instances_
TRIAL: {'metric_name': 'Tropicalized Homology Size / Circuit Size', 'metric_value': None, 'instances_

```

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_43419545.py", line 135, in <module>
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
                        ~~~~~^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_43419545.py", line 135, in <genexpr>
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
                        ~~~~~^

```

KeyError: 'seed'

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from verifying the conjecture's conditions. | next: Re-run the test with proper error handling to ensure it can complete and provide results.

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	16075
2	propose	ollama_remote	glm4:latest	0	0	14396
3	propose	ollama_remote	glm4:latest	0	0	13641
4	preregistration	ollama_remote	glm4:latest	0	0	9059
5	novelty	ollama_remote	glm4:latest	0	0	8106
6	novelty	ollama_remote	glm4:latest	0	0	9668
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13909
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8792
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13538
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13207
11	judge	ollama_remote	glm4:latest	0	0	11518

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 131909 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in *research/audit/1e7b5e62f37d.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/1e7b5e62f37d.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/1e7b5e62f37d.tar.gz` (if generated)